

```
def read_project_file(name: str) -> str:
    with open(name, "r") as f:
        return f.read()

# later, optionally, we define metadata about the function
@metric_registry.metadata(Automatic)
name="read_project_file",
function=read_project_file,
description="Reads a file from the project.",
parameters={
    "type": "string",
    "parameters": {
        "name": {"type": "string"}
    }
},
required= ["name"]
),
return read_file
```

In this code, if we change the function's parameters, we need to remember to update the parameter schema. If we modify what the function does, we need to remember to update the description. These manual steps are error-prone and can lead to confusion for both developers and the LLM.

Doing tool deconvolution, we can keep all this information together. Here's how we'll refactor our `FEEDBACK` agent:

[illegible]

- Documentation lives with the code it describes
- Changes to the function naturally lead to updates in documentation
- We get better IDE integration with docstring hints

- Parameter types, if any, are typed automatically
- Required parameters, if any, are detected from the function signature
- We don't need to manually maintain a variable table

- `file_operations`, for file-related tasks
- `read` and `write`, to specify the type of operation
- `system`, for administrative functions like bootstrapping

- Group related tools together
- Filter tools when creating agents
- Understand a tool's purpose at a glance

```
agent = Agent(
    goal=goal,
    agent_language=agent_util.all_languages(),
    action_registry=action_registry,
    generate_response=generate_response,
    environment=environment())
```

- Add new tools by simply decorating functions
- Modify existing tools without touching registration code
- Change tool organization by adjusting tags
- Remove tools by removing the decorator or changing its name

1. When the code loads, the decorators automatically register all tools in a central registry.
2. When we create an `AutofreeRegistry` with specific tags, it automatically loads the matching tools.